

FL-05: Agent Concepts and MCP Basics

General AI Fluency Track

NAME Leon	ASSIGNMENT FL-05	TRACK General AI Fluency	PROJECT FL-04 — Automation Workflow v2
---------------------	----------------------------	------------------------------------	--

1. What Is an Agent?

An AI agent is a system that is given a goal and can decide how to achieve that goal by selecting actions, using available tools, observing the results, and adjusting its approach when necessary. Unlike a simple chatbot that responds directly to a prompt, an agent can operate through multiple steps and make decisions during the process. The important characteristic is not simply that an AI performs several actions, but that the system can dynamically determine what actions are needed to reach the desired outcome.

An agent may follow a loop such as planning, acting, observing the result, and deciding what to do next. For example, if an agent is asked to create high-quality study notes, it might first inspect the source material, decide whether more information is required, gather additional information, create a draft, evaluate the draft, and revise it if the quality is not sufficient. The agent's path can change depending on what it observes during execution.

2. Workflow vs Agent

A workflow is a predefined sequence of steps designed by a human. Each step has a specific purpose and usually passes its output to the next step. Workflows are predictable and useful when the process is well understood. An agent, in contrast, has more autonomy. Instead of following only a fixed sequence, it can decide which actions or tools to use and whether it needs to repeat or change a step based on the results.

The key difference is therefore decision-making. A workflow follows a process that has been designed in advance, while an agent is capable of dynamically deciding how to proceed toward a goal. A workflow can still use AI and multiple tools without being an agent.

3. Classification of My FL-04 Workflow

My FL-04 Automation Workflow v2 is a **workflow, not an agent**. It follows a predefined three-stage pipeline: **Gather → Draft → Review**.

In the Gather stage, study material is uploaded to NotebookLM, which produces a source-grounded summary. The output is then passed to ChatGPT for the Draft stage, where structured exam-ready notes, definitions, flashcards, and quiz questions are created. Finally, the Review stage uses ChatGPT to improve formatting, grammar, readability, and completeness.

The stages and handoffs are defined in advance. The system does not independently decide whether it should gather more information, select a different tool, or repeat the review process. Human-designed instructions determine the sequence. Therefore, this is correctly classified as a workflow.

4. What Is MCP?

Model Context Protocol (MCP) is a standardized way for AI applications to connect with external systems and capabilities. It provides a common structure for AI models to interact with data sources and tools instead of being limited to information available inside the chat.

MCP can be understood as a connection layer between an AI application and external services. For example, an AI assistant connected to GitHub can access repository information and perform supported operations through the connector. This makes the AI more useful because it can work with real external information rather than only answering from the conversation context.

5. MCP's Three Primitives

The three important MCP primitives are tools, resources, and prompts.

- **Tools** — represent actions that an AI can perform, such as searching a repository, retrieving information, or interacting with an external service.
- **Resources** — represent information or data that an AI can access, such as files, documents, or repository content.
- **Prompts** — are reusable instructions or templates that guide how an AI should perform a task.

In simple terms, tools allow the AI to **do**, resources allow it to **access**, and prompts help it **follow instructions**.

6. My GitHub Connector Experiment

For the practical part of this assignment, I used a GitHub connector with Claude and connected it to my FL-04-Automation-Workflow-v2 repository. I performed three tasks that required access to external repository information.

First, I asked Claude to inspect the repository structure and identify its folders and files. Second, I asked Claude to open and analyze the README.md file, which allowed it to retrieve information about the project's purpose, workflow stages, and time savings. Third, I asked Claude to search for references to NotebookLM and explain its role in the Gather → Draft → Review pipeline.

These tasks demonstrated that the connector allowed Claude to work with information stored in my GitHub repository rather than relying only on the text already present in the chat.

7. Turning FL-04 Into an Agent

To turn my FL-04 workflow into an agent, the system would need more autonomy and dynamic decision-making. Instead of always following Gather → Draft → Review in exactly the same way, an agent could evaluate the quality of each intermediate result and decide what to do next.

For example, after gathering information, the agent could determine whether the source summary is complete enough. If important information is missing, it could decide to gather more information or use another available tool. After drafting the notes, it could evaluate the result and decide whether another revision is necessary. The agent would continue this process until the output meets defined quality criteria.

8. Concrete Agent Upgrade

One concrete upgrade for my FL-04 pipeline would be a **dynamic quality-check and decision loop**. The agent would inspect the gathered information and generated notes, identify missing or weak areas, and autonomously decide whether to gather more information, revise the draft, repeat the review, or produce the final answer.

This would change the system from a fixed Gather → Draft → Review workflow into a goal-driven process where the AI can choose its next action based on intermediate results. Human review would still be important for final accuracy, especially for technical content, diagrams, scanned documents, and OCR errors, but the system would have significantly more agent-like behavior.

9. Evidence of GitHub Connector Tasks

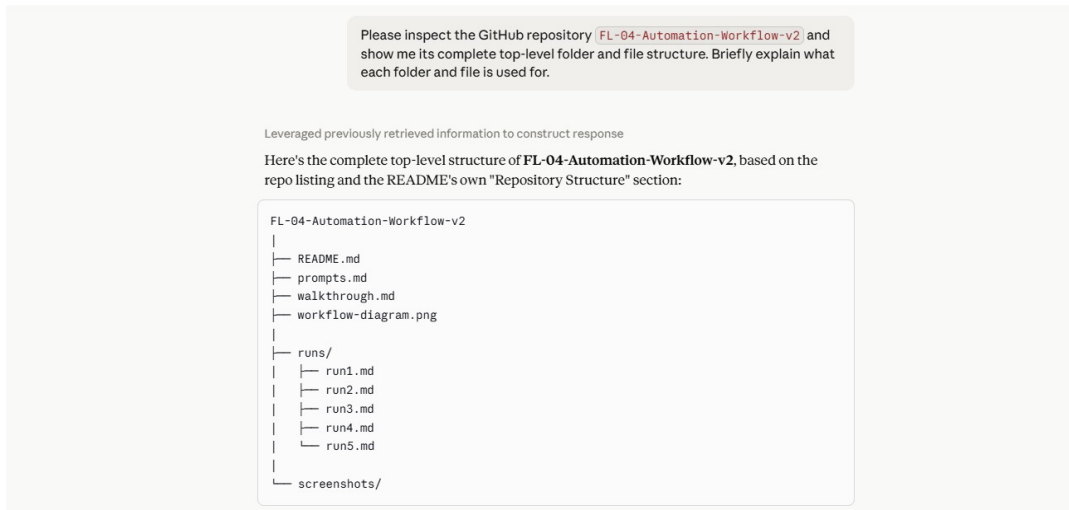


Figure 1. Claude inspecting the FL-04-Automation-Workflow-v2 repository structure.

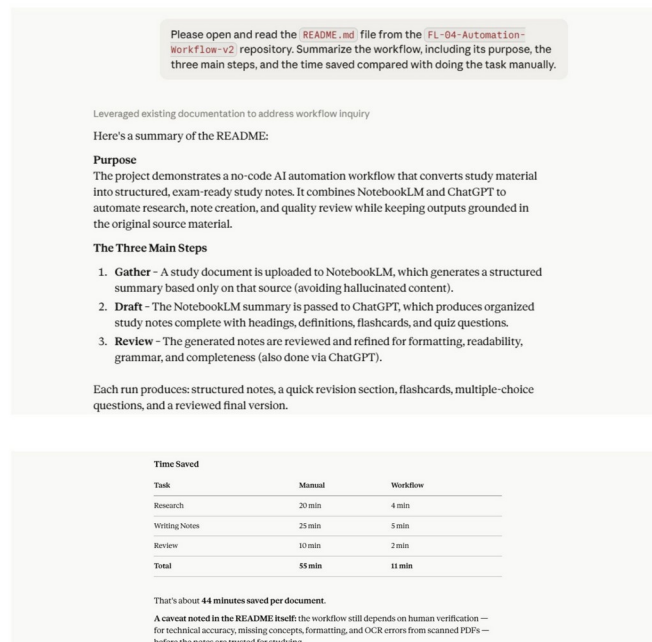


Figure 2. Claude analyzing README.md for project purpose and workflow stages.

Search the `FL-04-Automation-Workflow-v2` GitHub repository for references to "NotebookLM". Find where NotebookLM is mentioned and explain how it is used in the Gather → Draft → Review workflow. Also identify one potential failure point in this workflow.

Where NotebookLM is mentioned (in `README.md`)

1. **Project Overview** — the workflow combines NotebookLM and ChatGPT to automate research, note creation, and quality review while maintaining source-grounded outputs. `github`
2. **Tools Used** — listed as "NotebookLM - Research and source-grounded summarization" `github`
3. **Step 1 (Gather)** — a study document is uploaded to NotebookLM, which generates a structured summary using only the uploaded source. `github`

How it fits into Gather → Draft → Review

- **Gather (NotebookLM's stage):** NotebookLM ingests the raw study document and produces a summary restricted to that source's content — it's not meant to draw on outside knowledge. This summary is the sole output of this stage.
- **Draft (ChatGPT):** That NotebookLM summary — not the original document — becomes ChatGPT's only input. ChatGPT expands it into structured notes, definitions, flashcards, and quiz questions.
- **Review (ChatGPT):** ChatGPT (in a separate pass) checks and polishes what it just drafted for formatting, grammar, and completeness.

Figure 3. Claude searching for NotebookLM references within the Gather → Draft → Review pipeline.